

Auto Zoom grandma3 plugin

Installation

1. Download the files `autozoom-grandma3.lua` and `autozoom-grandma3.xml`
2. Move the downloaded files to :
 - Either a USBStick in folder : `grandMA3\gma3_library\datapools\plugins`
 - Either to your onPC datapools folder :
`C:\ProgramData\MALightingTechnology\gma3_library\datapools\plugins`
3. Import the plugin in grandma3 (help : [Doc](#))

Usage

Preparing the console

Make sure that XYZ is enabled for all fixture types that you want to use autozoom with.

Also make sure that the zoom and iris physical ranges (of the fixture type) correspond to the optical parameters given by the constructor on the fixture's datasheet.

Importing the macros

- Launch the plugin to import all the commands. All the commands that this plugin creates need to start by `AZ:`. To launch a command, simply type `Lua "AZ:<command>(<args>)"` in the command line or in a macro.
- You can create all the available macros by typing `Lua "AZ:CreateMacros(<starting-index>)"`

Reading the patch

- When the plugin starts, it automatically scan the patch to see where the fixtures are in 3D space, and their Zoom/Iris physical ranges.
- If you change the fixtures position, or change the physical ranges for any fixture, please rescan the patch by executing `Lua "AZ:ScanPatch()"`

Enabling Autozoom on a fixture

- Supposed you have a tracking sequence, in which the position of the fixture is set to the marker.
- Store in the same cue the zoom and iris values at minimum.
- Create 2 sequences 'AZ_ZOOM_<fixture_id>' and 'AZ_IRIS_<fixture_id>' with zoom and iris at max (manually or automatically using `Lua "AZ:CreateAZSequences(<fid>)"`)
- Enable the fixture to a certain beam size : `Lua "AZ:EnableFixture(<fixture_id>, <marker_fid>, <beamSize>)"`

Note on using automatic sequence creation :

Automatic creation creates fixtures either in a new datapool called "AZ", or in the current datapool you're in. You can enable/disable the use of the "AZ" Datapool using `Lua "AZ:EnableDatapool()"` or `Lua "AZ:DisableDatapool()"` It works on fixtures that are enabled only (because in the future, it will create also the tracking sequence, with XYZ 0, Zoom and Iris min, and marker set)

Disabling a fixture

- To disable Autozoom on a fixture, simply launch / type `Lua "AZ:DisableFixture(<fixture_id>)"`
- If you want to disable AutoZoom on all fixtures at once, type `Lua "AZ:DisableAllFixtures()"`

Using a fader for beamSize

- To use a fader as Beam Size Fader, you can either :
 - Use one for each fixture ex : (AZ_SIZE_101, AZ_SIZE_102, ...)
 - Use one global fader for all fixture : "AZ_SIZE"
- To enable Size Fader for a fixture, type `Lua "AZ:EnableSizeFader(<fid>)"`
- To go back to fixed value, use `Lua "AZ:DisableSizeFader(<fid>)"`
- To enable the "global" fader, use : `Lua "AZ:EnableGlobalSizeFader()"`
- To disable the "global" fader, use : `Lua "AZ:DisableGlobalSizeFader()"`
- The faders need to have a range set, to remap the fader value (0->100) to a beamSize in meters (for exemple : 1m -> 5m). To change this range, use `Lua "AZ:SetSizeFaderRange(<min>, <max>)"`

Get the current status

- You can print the current status of the fixtures (which one is following which marker, at which beam_size) using `Lua "AZ:GetFixturesStatus()"`

Developping

This plugin uses [TypescriptToLua](#) and [GrandMA3-TS-Types](#).

To install them, use node js, and type `npm install` in folder.

You can build using `npm run build` or use `npm run dev` to automatically build the plugin on code change.